

Tema 3.5: Autenticación y Autorización en APIs RESTful (Spring Security)

Resumen

Este tema analiza los pilares fundamentales de la seguridad en el desarrollo back-end con Spring Boot 3.x y Spring Security 6: la **Autenticación** (verificación explícita de la identidad del usuario) y la **Autorización** (evaluación y gestión de privilegios de acceso basados en roles o autoridades). Se detalla paso a paso la creación y administración explícita de infraestructuras de seguridad, incluyendo el hash de contraseñas con `BCryptPasswordEncoder`, la implementación de `UserDetailsService`, la configuración de la cadena de filtros `SecurityFilterChain` y el control de acceso a nivel de método con `@PreAuthorize`.

Concepto Principal (El 'Qué')

La seguridad de software se sostiene sobre dos conceptos interrelacionados pero técnicamente diferenciados:

1. **Autenticación (Authentication):** Es el proceso mediante el cual el sistema verifica de forma inequívoca la identidad planteada por una entidad (usuario, dispositivo o servicio). Responde a la pregunta: *¿Quién es usted?* La autenticación comúnmente requiere credenciales como usuario/contraseña, certificados digitales o factores biométricos.
2. **Autorización (Authorization):** Es el proceso posterior mediante el cual el sistema evalúa si el sujeto previamente autenticado posee los derechos o permisos suficientes para ejecutar una acción específica o acceder a un recurso determinado. Responde a la pregunta: *¿Tiene usted permiso para realizar esta operación?*

En la pila tecnológica de Java, **Spring Security** es el framework estándar de la industria. Su arquitectura interna se basa en una cadena de filtros de servlet (`FilterChain`) que interceptan las peticiones HTTP antes de que alcancen la capa de controladores.

Componentes Clave de la Arquitectura de Spring Security:

- **SecurityContextHolder**: Almacena el `SecurityContext` actual de la aplicación, el cual contiene el objeto `Authentication` del usuario autenticado en el hilo de ejecución (`ThreadLocal`).
- **Authentication**: Representa la identidad autenticada, incluyendo sus credenciales, detalles y la colección de permisos concedidos (`GrantedAuthority`).
- **GrantedAuthority**: Permiso individual o rol asignado al usuario (ejemplo: `ROLE_ADMIN`, `READ_PRIVILEGE`).
- **UserDetailsService**: Interfaz de Spring Security encargada de recuperar los datos del usuario desde la fuente de persistencia (Base de Datos).
- **PasswordEncoder**: Interfaz para realizar el hash unidireccional y verificación segura de contraseñas.

Analogía (La Intuición)

Imagine el proceso de ingreso a un edificio gubernamental de máxima seguridad:

1. **Autenticación**: En la recepción principal, usted presenta su documento de identidad físico (cédula o pasaporte). El oficial de seguridad verifica la validez del documento y confirma que su fotografía coincide con su rostro. Una vez verificado, se le entrega un carné de visitante. Ha sido *autenticado*.
2. **Autorización**: Usted intenta ingresar al laboratorio de servidores en el piso 5. El lector biométrico de la puerta escanea su carné y consulta la base de datos de permisos. Aunque el sistema reconoce quién es usted (autenticación exitosa), la puerta no se abre porque su carné únicamente tiene asignado el rol de "Visitante General" y no el permiso de "Administrador de Red". Se le ha denegado la *autorización*.

Creación y Gestión Explícita de la Seguridad (El 'Cómo')

A continuación se detalla cómo crear y administrar explícitamente el sistema de seguridad en Spring Boot 3.x:

1. Entidades de Dominio (Usuario y Rol)

```
package cr.ac.ucr.paraíso.ie.expresofast.domain;

import jakarta.persistence.*;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "Usuario")
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "usuario_id")
    private Integer id;

    @Column(nullable = false, unique = true, length = 50)
    private String username;

    @Column(name = "password_hash", nullable = false, length = 255)
    private String password;

    @Column(nullable = false)
    private boolean activo;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(
        name = "UsuarioRol",
        joinColumns = @JoinColumn(name = "usuario_id"),
        inverseJoinColumns = @JoinColumn(name = "rol_id")
    )
    private Set<Rol> roles = new HashSet<>();

    // Constructores, Getters y Setters
    public Integer getId() { return id; }
    public String getUsername() { return username; }
    public String getPassword() { return password; }
    public boolean isActive() { return activo; }
    public Set<Rol> getRoles() { return roles; }
    public void setPassword(String password) { this.password = password; }
}
```

```

package cr.ac.ucr.paraíso.ie.expresofast.domain;

import jakarta.persistence.*;

@Entity
@Table(name = "Rol")
public class Rol {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "rol_id")
    private Integer id;

    @Column(name = "nombre_rol", nullable = false, unique = true)
    private String nombreRol; // Ejemplo: 'ROLE_ADMIN', 'ROLE_OPERADOR'

    public Integer getId() { return id; }
    public String getNombreRol() { return nombreRol; }
}

```

2. Servicio Personalizado de Carga de Usuarios (CustomUserDetailsService)

```

package cr.ac.ucr.paraíso.ie.expresofast.security;

import cr.ac.ucr.paraíso.ie.expresofast.domain.Usuario;
import cr.ac.ucr.paraíso.ie.expresofast.data.UsuarioRepository;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.*;
import org.springframework.stereotype.Service;

import java.util.Set;
import java.util.stream.Collectors;

@Service
public class CustomUserDetailsService implements UserDetailsService {

    private final UsuarioRepository usuarioRepository;

    public CustomUserDetailsService(UsuarioRepository usuarioRepository) {
        this.usuarioRepository = usuarioRepository;
    }

    @Override

```

```

    public UserDetails loadUserByUsername(String username)
        throws UsernameNotFoundException {

        Usuario usuario = usuarioRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException(
                "Usuario no encontrado: " + username));

        Set<GrantedAuthority> authorities = usuario.getRoles().stream()
            .map(rol -> new SimpleGrantedAuthority(rol.getNombreRol()))
            .collect(Collectors.toSet());

        return new User(
            usuario.getUsername(),
            usuario.getPassword(),
            usuario.isActivo(),
            true, true, true,
            authorities
        );
    }
}

```

3. Configuración Explícita de Seguridad (SecurityConfig)

```

package cr.ac.ucr.paraíso.ie.expresofast.config;

import cr.ac.ucr.paraíso.ie.expresofast.security.CustomUserDetailsService;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
public class SecurityConfig {

```

```

    private final CustomUserDetailsService userDetailsService;

    public SecurityConfig(CustomUserDetailsService userDetailsService) {
        this.userDetailsService = userDetailsService;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(10); // Hash seguro con BCrypt
    }

    @Bean
    public AuthenticationManager authenticationManager(
        AuthenticationConfiguration authConfig) throws Exception {
        return authConfig.getAuthenticationManager();
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {
        http
            .csrf(csrf -> csrf.disable()) // Deshabilitado para APIs REST stateless
            .cors(Customizer.withDefaults())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/envios/**")
                    .hasAnyRole("ADMIN", "OPERADOR", "CONDUCTOR")
                .requestMatchers(HttpMethod.POST, "/api/envios/**")
                    .hasAnyRole("ADMIN", "OPERADOR")
                .requestMatchers("/api/vehiculos/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .httpBasic(Customizer.withDefaults()); // 0 JWT Filter en Tema 3.6

        return http.build();
    }
}

```

4. Autorización a Nivel de Método en Controladores REST

```

package cr.ac.ucr.paraíso.ie.expresofast.controller;

import cr.ac.ucr.paraíso.ie.expresofast.business.EnvioService;
import org.springframework.http.ResponseEntity;

```

```

import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/admin/flotas")
public class FlotaController {

    private final EnvioService envioService;

    public FlotaController(EnvioService envioService) {
        this.envioService = envioService;
    }

    @DeleteMapping("/{vehiculoId}")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<Void> darDeBajaVehiculo(
        @PathVariable Integer vehiculoId) {

        envioService.darDeBajaVehiculo(vehiculoId);
        return ResponseEntity.noContent().build();
    }
}

```

Explicación Paso a Paso del Código:

1. **CustomUserDetailsService**: Implementa la interfaz de Spring Security para consultar la tabla Usuario en SQL Server. Mapea la colección de entidades `Ro1` a objetos `SimpleGrantedAuthority`.
2. **BCryptPasswordEncoder (10)**: Configura un encriptador de contraseñas basado en el algoritmo BCrypt con un factor de costo (*work factor*) de 10. BCrypt aplica una sal aleatoria (*salt*) automáticamente a cada hash, previniendo ataques de tablas arcoíris (*rainbow tables*).
3. **SecurityFilterChain**: Bean donde se registran explícitamente las reglas de acceso por URL. `requestMatchers("/api/auth/**").permitAll()` permite acceso público a endpoints de autenticación, mientras que `hasRole("ADMIN")` exige que el usuario posea la autoridad `ROLE_ADMIN`.
4. **@EnableMethodSecurity & @PreAuthorize**: Habilita la seguridad declarativa en métodos de controlador o servicios. La anotación `@PreAuthorize("hasRole('ADMIN')")` evalúa la expresión SpEL (*Spring Expression Language*) antes de ejecutar el cuerpo del método.

Trampas Comunes

Almacenamiento de Contraseñas en Texto Plano:

- *Error:* Guardar las contraseñas directamente como cadenas legibles en la base de datos o utilizar algoritmos obsoletos como MD5 o SHA-1.
- *Solución:* Utilizar obligatoriamente `BCryptPasswordEncoder` o `Argon2PasswordEncoder`. Nunca almacenar ni registrar en logs contraseñas en texto plano.

Confusión entre Roles y Autoridades (hasRole vs hasAuthority):

- *Error:* Definir en la base de datos el nombre del rol como "ADMIN" y utilizar `hasRole("ADMIN")`, lo cual falla porque Spring Security busca automáticamente el prefijo `ROLE_` (`ROLE_ADMIN`).
- *Solución:* Almacenar los roles en base de datos con la convención `ROLE_NOMBRE` o utilizar `hasAuthority("ADMIN")` si se prefieren permisos finos sin prefijo.

Bloqueo de Peticiones por CSRF en APIs REST Stateless:

- *Error:* Dejar habilitada la protección CSRF (*Cross-Site Request Forgery*) por defecto en aplicaciones REST que utilizan autenticación stateless basada en tokens, lo que provoca rechazos `403 Forbidden` en peticiones `POST`, `PUT` o `DELETE`.
- *Solución:* Deshabilitar explícitamente CSRF (`http.csrf(csrf -> csrf.disable())`) únicamente cuando la API sea stateless y no dependa de cookies de sesión tradicionales del navegador.

Pregunta de Reflexión

En sistemas empresariales complejos: ¿Cuáles son las ventajas y desventajas de migrar de un modelo **RBAC** (Control de Acceso Basado en Roles) hacia un modelo **ABAC** (Control de Acceso Basado en Atributos) donde la autorización depende de variables en tiempo de ejecución (como la hora actual, la ubicación IP del usuario y el monto de la transacción)?

